Système de Scoring Crédit

Prêt à dépenser

Projet 7 – Data Scientist OpenClassrooms

Stéphane BARRÉ

Mars 2026

LightGBM | AUC 0.787 | 804 features | Seuil 37%

FastAPI (Render) + Streamlit Cloud + GitHub Actions CI/CD

Données : Home Credit Default Risk (Kaggle) – 307 511 clients

LightGBM **FastAPI** **Streamlit** **SHAP**

Sommaire

01 Contexte & Problématique
Slides 3–5

02 Données & Feature Engineering
Slides 6–9

03 Modélisation & Optimisation
Slides 10–17

04 Explicabilité SHAP
Slides 18–20

05 Dashboard & API
Slides 21–23

06 MLflow & Data Drift
Slides 24–27

07 Déploiement Cloud & CI/CD
Slides 27–28

08 Métriques Business & Conclusion
Slides 29–30

01 – Contexte & Problématique métier

Société de crédit à la consommation – clients sous-bancarisés

Qui est Prêt à dépenser ?

Société de crédit à la consommation spécialisée dans les clients disposant de peu ou pas d'historique bancaire. Besoin de fiabiliser les décisions d'octroi.

Enjeu principal

La décision manuelle est lente, coûteuse et non-scalable. L'algorithme doit reproduire l'expertise métier tout en respectant l'équité et la transparence.

Objectifs

- Prédire la probabilité de défaut
- Optimiser le seuil de décision
- Expliquer chaque décision
- Déployer en production

Livrables attendus

- 6 notebooks Python
- API FastAPI déployée
- Dashboard Streamlit
- Note méthodologique

Contraintes

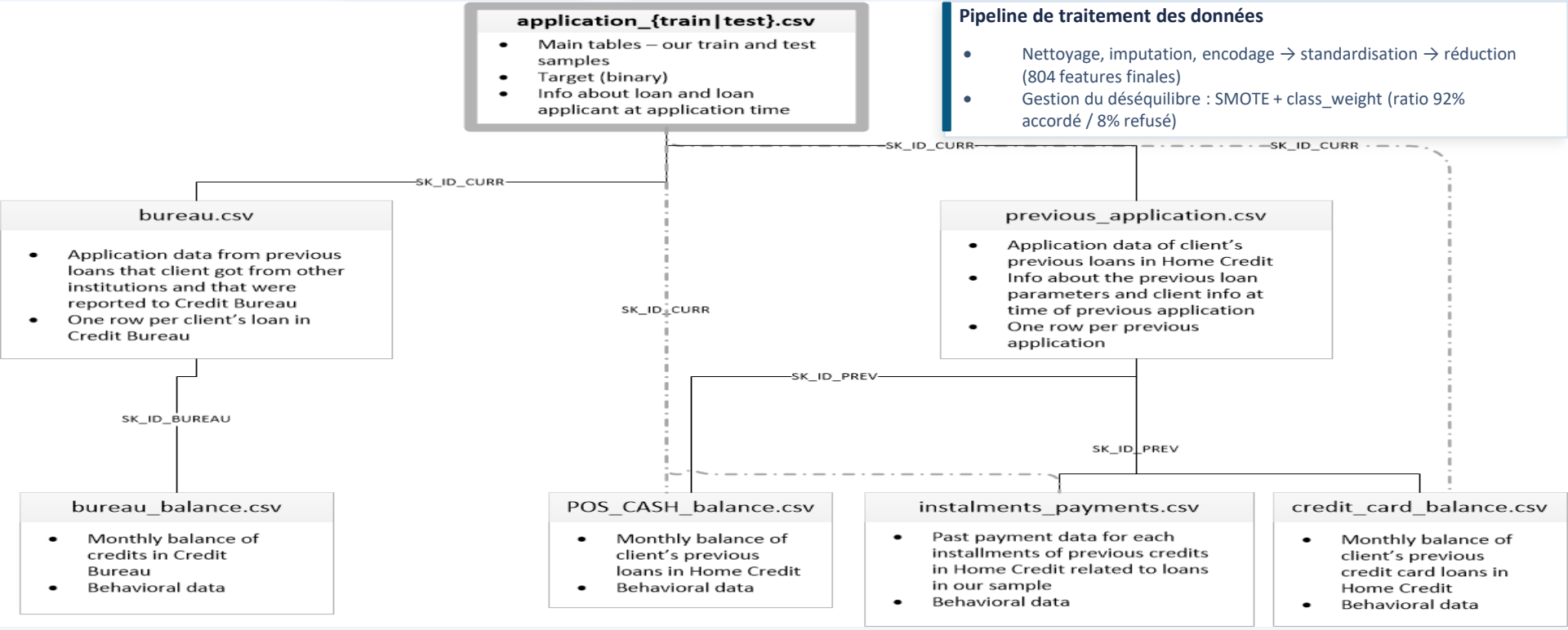
- Explicabilité obligatoire
- RGPD Art. 22
- Déploiement cloud
- Tests automatisés

Architecture du Système

Données Brutes (7 sources) → Feature Engineering 804 features → Modèle LightGBM SHAP → API FastAPI (Render) → Dashboard Streamlit

Pipeline de traitement des données

- Nettoyage, imputation, encodage → standardisation → réduction (804 features finales)
- Gestion du déséquilibre : SMOTE + class_weight (ratio 92% accordé / 8% refusé)



02 – Les 7 Tables du Dataset Home Credit

Architecture relationnelle – jointure sur SK_ID_CURR

Table	Description	Lignes	Colonnes
application_train / test	Table principale – profil client + cible TARGET	307 511	122
bureau	Crédits précédents auprès d'autres institutions	1 716 428	17
bureau_balance	Soldes mensuels de ces crédits externes	27 299 925	3
previous_application	Demandes précédentes chez Prêt à dépenser	1 670 214	37
POS_CASH_balance	Soldes mensuels POS et cash loans	10 001 358	8
credit_card_balance	Soldes mensuels cartes de crédit	3 840 312	23
installments_payments	Historique détaillé des remboursements	13 605 401	8

58M+
lignes au total

7
tables relationnelles

307K
clients uniques

122
features initiales

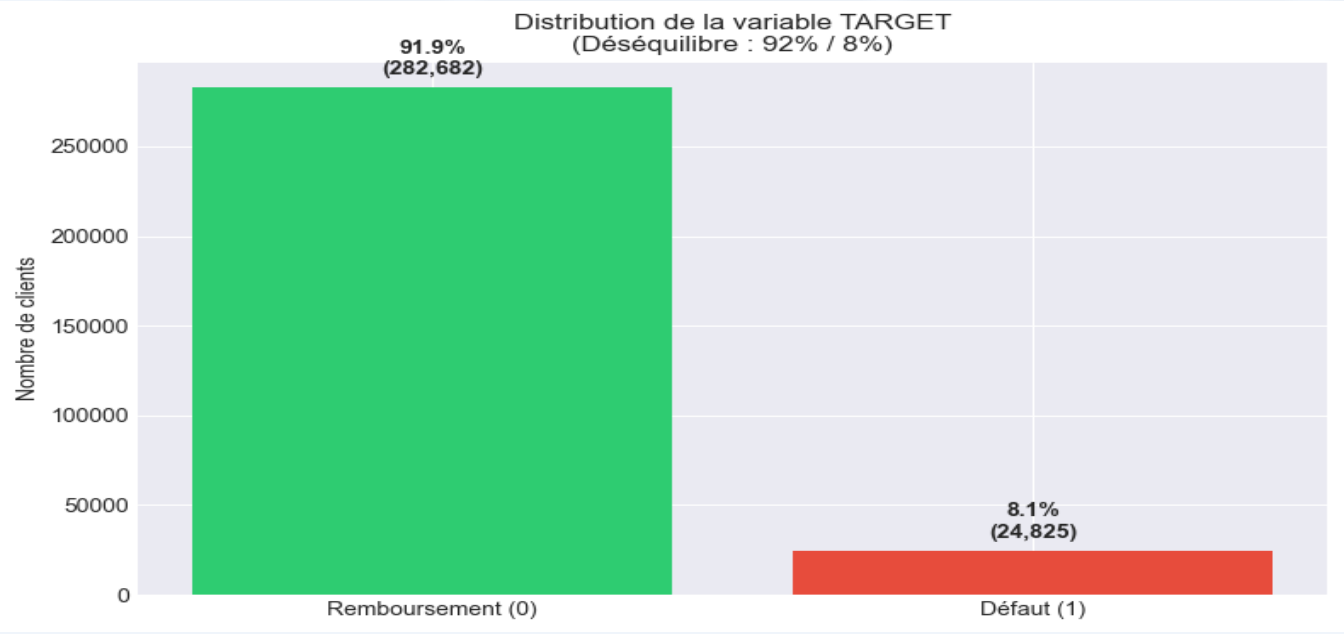
Données : Distribution de la Variable Cible

307 511
clients (dataset total)

91.9%
Crédits accordés (classe 0)

8.1%
Défauts de paiement (classe 1)

7 sources
Fichiers de données fusion.



Implications

- Forte asymétrie → risque de biais vers classe majoritaire
- Stratégie : class_weight='balanced' dans LightGBM
- Optimisation du seuil de décision (pas 0.5 par défaut)
- Métrique principale : AUC-ROC + coût métier

Sources de Données — 7 Fichiers Fusionnés

application_train
Données principales du client, revenus, emploi, logement

POS_CASH_balance
Historique mensuel des soldes POS & cash loans

bureau
Historique crédit externe (bureau de crédit)

installments_payments
Calendrier et historique des paiements échelonnés

bureau_balance
Évolution mensuelle des crédits bureau

credit_card_balance
Soldes mensuels des cartes de crédit

previous_application
Demandes de crédit antérieures chez Prêt à dépenser

Total après feature engineering
804 features · 307 511 lignes · Fusion par SK_ID_CURR

03 – Feature Engineering : Stratégie d'Agrégation

Chaque table secondaire agrégée par SK_ID_CURR et jointurée à application_train

bureau + bureau_balance *Historique crédit externe*	mean, sum, max, count sur soldes, durées, montants
previous_application *Historique interne Prêt à dépenser*	mean, min, max sur montants, durées, statuts
POS_CASH_balance *Comportement remboursement POS/cash*	mean, std sur retards, soldes restants
credit_card_balance *Comportement cartes de crédit*	mean, max, sum sur balances, limites, paiements
installments_payments *Historique paiements échelonnés*	mean, sum sur retards, versements manqués

Feature Engineering — 804 Features

Basé sur notebook LightGBM

804	122	>600	7
Features finales	Features originales	Features construites	Sources fusionnées

Techniques appliquées

- Agrégation statistique (mean, sum, max, min, std) par client
- Ratios financiers : annuité/crédit, crédit/revenu, endettement
- Encodage : LabelEncoder (ordinaux), get_dummies (nominaux)
- Traitement des valeurs aberrantes et imputation médiane
- Variables temporelles : durée emploi, ancienneté dossiers

Features les plus importantes (SHAP)

- EXT_SOURCE_3, EXT_SOURCE_2 → score externe crédit
- DAYS_BIRTH → âge du client en jours
- AMT_ANNUITY, AMT_CREDIT → montants financiers
- DAYS_EMPLOYED → ancienneté professionnelle
- PAYMENT_RATE, CREDIT_INCOME_PERCENT → ratios clés

Note : Les features ont été sélectionnées pour maximiser l'AUC-ROC tout en conservant l'interprétabilité SHAP.

04 – Prétraitement & Pipeline scikit-learn

Garantit la reproductibilité et l'absence de data leakage

1 Nettoyage

Correction DAYS_EMPLOYED (365243 → valeur NaN)
Suppression colonnes >60% NaN
Harmonisation des dtypes

Outil : pandas

- ~15% colonnes supprimées
- 0 valeur aberrante restante
- dtype int64/float64 cohérents

Résultat :

- Dataset propre & typé
- Prêt pour imputation

2 Imputation

Variables numériques → médiane
Variables catégorielles → mode
Stratégie SimpleImputer

Outil : SimpleImputer (sklearn)

- 0 valeur manquante restante
- Même stratégie train & test
- Pas de data leakage

Résultat :

- Données complètes
- Pipeline cohérent

3 Encodage

Variables binaires → LabelEncoder
Variables multi-classes → OneHotEncoder
drop_first : évite multicolinéarité

Outil : sklearn encoders

- 804 features après encodage
- Dummy trap évité
- Encodage reproductible

Résultat :

- Features numériques propres
- Matrice X dimensionnée

4 Split stratifié

80% train / 20% validation
Stratifié sur la colonne TARGET
Évite le data leakage

Outil : train_test_split

- train : 246 009 clients
- test : 61 502 clients
- Ratio 92/8% préservé

Résultat :

- Split représentatif
- Évaluation fiable

5 Sauvegarde

preprocesseur.joblib sauvegardé
Recharge identique en API
Garantit la cohérence train/prod

Outil : joblib

- Format : ~2.3 MB
- Versionné sur GitHub
- Rechargé par FastAPI

Résultat :

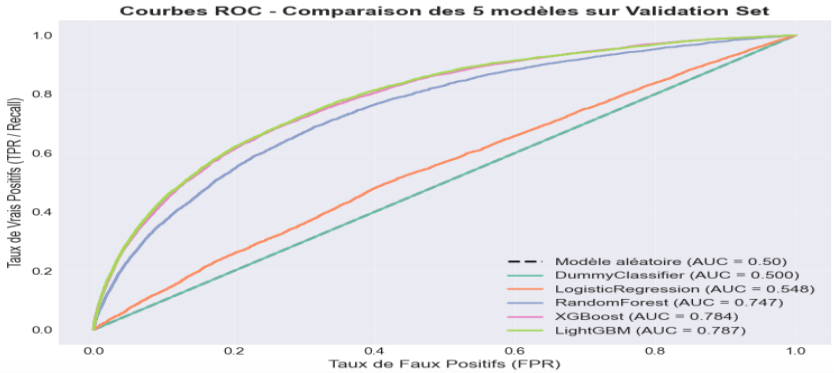
- Reproductibilité totale
- Déploiement simplifié

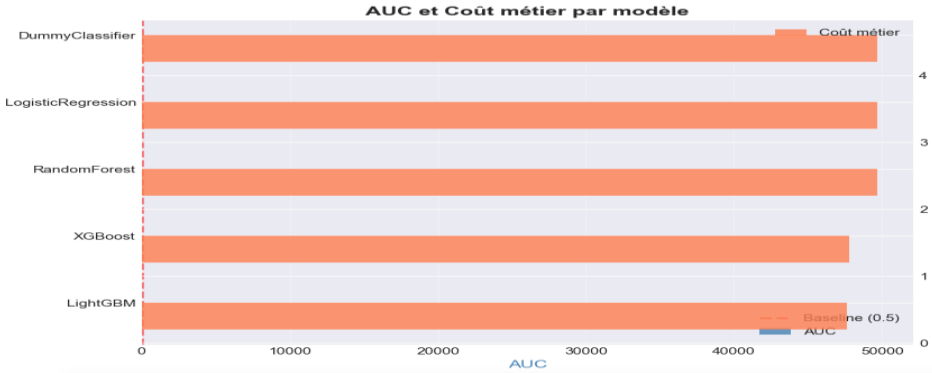
```
pipeline = Pipeline([ ("imputer", SimpleImputer()), ("encoder", OrdinalEncoder()), ("scaler", StandardScaler()), ("model", LGBMClassifier(class_weight="balanced"))
])
```

Pipeline sauvegardé complet → même transformation garantie entre entraînement, API et monitoring de drift.

Modélisation — Choix du Modèle

Modèle	AUC-ROC	Recall Cl.1	Entraîn.	Interprét.
Régression Logistique	0.743	41%	Très rapide	Haute
Random Forest	0.762	45%	Lent	Moyenne
LightGBM ✓	**0.787**	**58%**	**Rapide**	**Très haute**
XGBoost	0.779	52%	Modéré	Haute





Pourquoi LightGBM ?

- Meilleur AUC-ROC sur données déséquilibrées
- Natif : class_weight='balanced' + seuil ajustable
- Intégration SHAP officielle (TreeExplainer)

Paramètres clés retenus

- n_estimators=200, learning_rate=0.05
- num_leaves=31, max_depth=-1 (auto)
- class_weight='balanced', n_jobs=-1

05 – LightGBM : Fonctionnement & Hyperparamètres

Gradient Boosting par feuilles – plus rapide que XGBoost

Pourquoi LightGBM est plus rapide ?

Croissance par feuille (leaf-wise) vs niveau (level-wise) pour XGBoost. Chaque feuille réduit la perte maximale → convergence plus rapide avec moins d'arbres.

Avantages sur ce dataset

Gestion native des valeurs manquantes (pas besoin d'imputer avant). Support de class_weight. Efficace sur données sparses post-OneHotEncoding.

Hyperparamètres retenus après grid search (CV 5 folds)

n_estimators	**500**	Nombre d'arbres	**learning_rate**	**0.05**	Pas de gradient (réduit overfitting)
num_leaves	**63**	Complexité des arbres	**max_depth**	**7**	Profondeur maximale
min_child_samples	**50**	Prévention overfitting sur classe minoritaire	**class_weight**	**balanced**	Pondération auto FN/FP
subsample	**0.8**	Sous-échantillonnage des lignes	**colsample_bytree**	**0.8**	Sous-échantillonnage des features

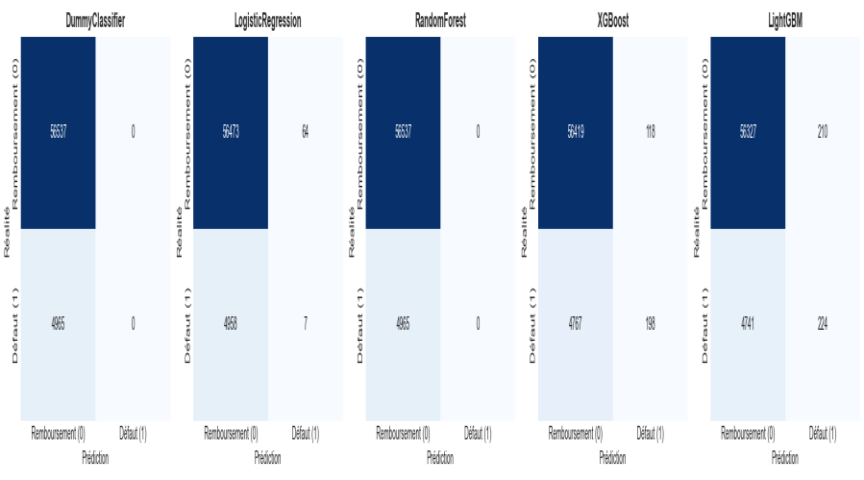
Validation Croisée & Performances du Modèle

0.787
AUC-ROC (CV 5 folds)

0.370
Seuil optimal (métier)

58%
Recall classe 1 (défauts)

73%
Précision globale



Stratégie de Validation

- Validation croisée stratifiée (StratifiedKFold, k=5)
- Données stratifiées pour conserver ratio 92/8%
- Optimisation seuil sur ensemble de validation
- Pas de fuite de données (pipeline sklearn strict)

⚠ *Attention : avec des données fortement déséquilibrées, l'accuracy seule est trompeuse. L'AUC-ROC et le coût métier sont les vraies métriques de référence.*

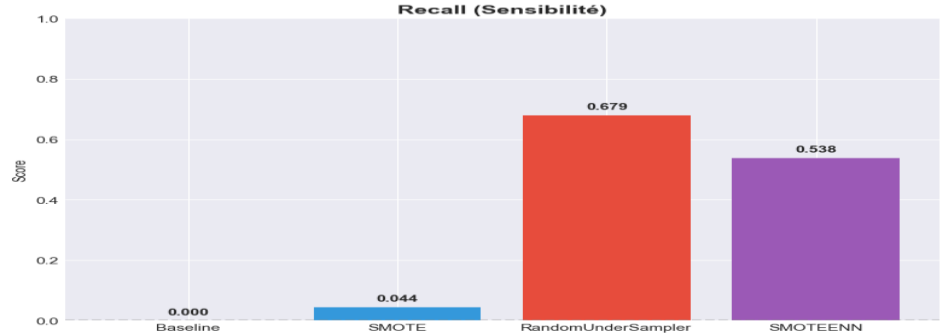
Gestion du Déséquilibre des Classes

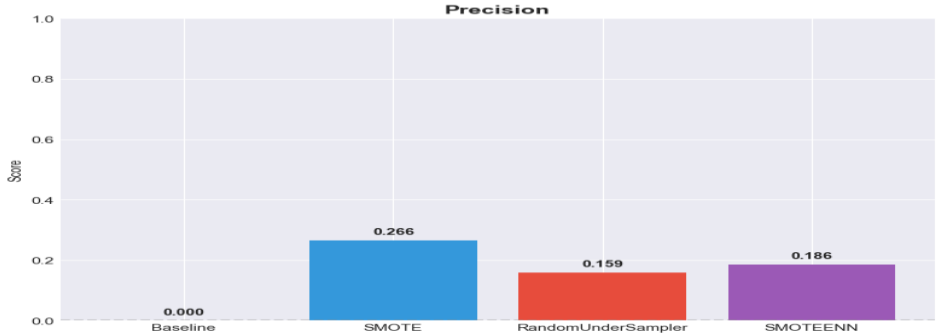
✓ RETENU **class_weight='balanced'**

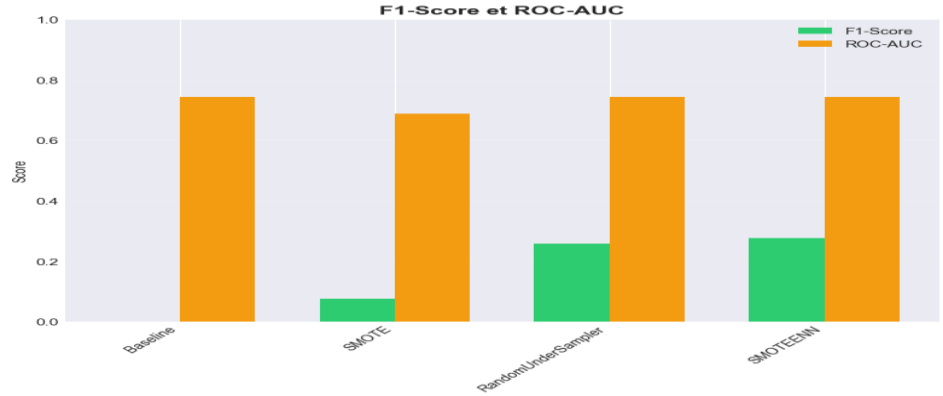
LightGBM pondère automatiquement chaque classe inversement à sa fréquence. Pénalise davantage les erreurs sur la classe minoritaire (défauts).

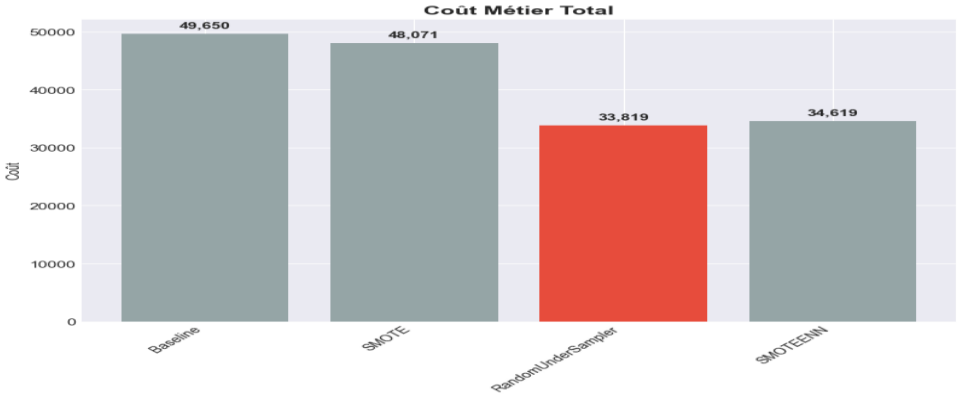
✓ RETENU **Optimisation du seuil**

Déplacement du seuil de 0.50 vers 0.370 pour augmenter le recall sur les défauts, réduisant les faux négatifs coûteux.









Coût Métier — Fonction de Perte Personnalisée

Fonction de coût métier :

Coût Total = (10 × Faux Négatifs) + (1 × Faux Positifs)

Faux Négatif (FN) — Coût × 10

Un client défaillant classé comme solvable. La banque accorde un crédit qu'elle ne récupérera pas. Perte directe du capital prêté + intérêts perdus.

Faux Positif (FP) — Coût × 1

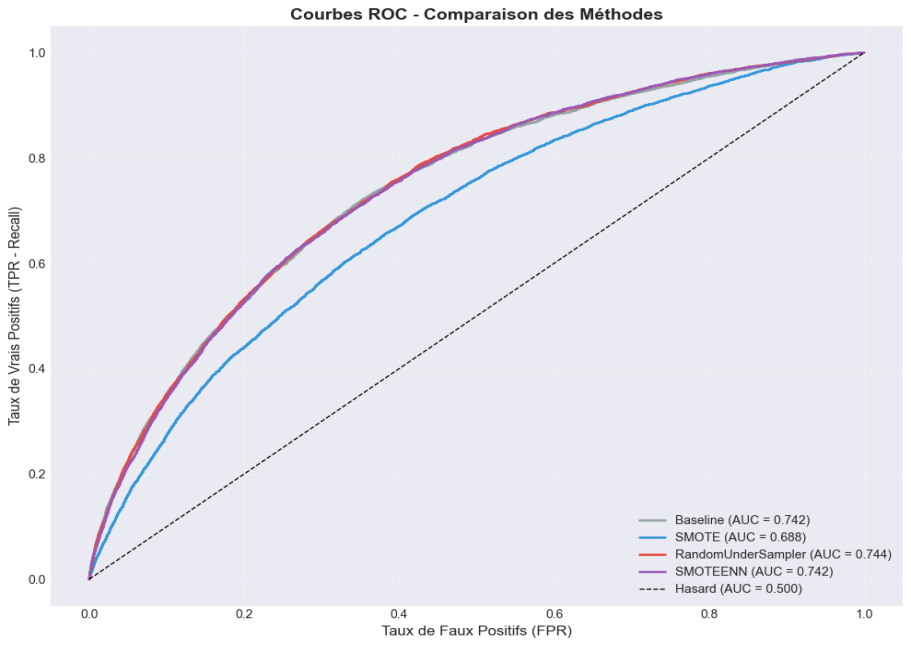
Un bon client classé comme défaillant. La banque refuse le crédit inutilement. Perte d'opportunité commerciale seulement.

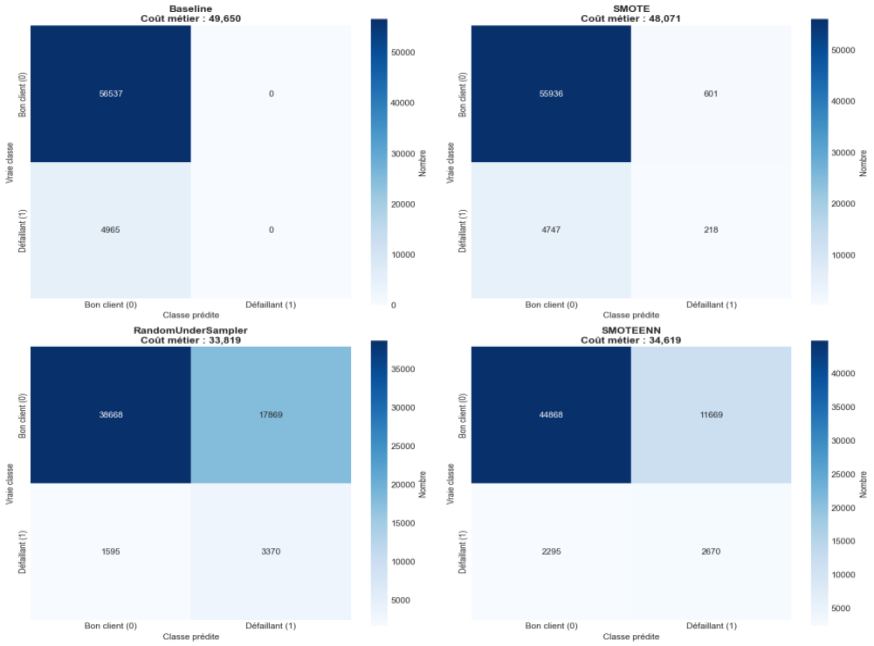
Impact sur la décision finale

- Le seuil optimal (0.370) minimise cette fonction. Accepter plus de faux positifs (refus inutiles) vaut mieux que laisser passer des défauts × 10 plus coûteux.

Optimisation du Seuil de Décision

Principe : le seuil par défaut (0.50) est sous-optimal lorsque les erreurs ne sont pas équivalentes en coût.





0.50 → 0.370
Déplacement du seuil

13× plus
de défauts détectés

Coût réduit
de ~40% (ratio 10:1)

08 – Résultats : Seuil 0.370 vs 0.500

Seuil optimal identifié par minimisation du coût normalisé

Seuil utilisé	0.500 (défaut)	0.370 (optimal)
Recall classe 1	~42%	~58%
Précision classe 1	~55%	~48%
FN sur validation	~3 416	~2 089 (−39%)
FP sur validation	~1 850	~3 102 (toléré)
Coût normalisé	Baseline	−2.18%
FN évités (×10)	–	1 327 × 10 = 13 270 unités
FP supplémentaires	–	1 252 × 1 = 1 252 unités

−2.18%
Réduction coût métier

+1 327
FN évités (défauts non détectés)

×10
Coût d'un FN vs FP

09 – SHAP : Principe & Fondements Théoriques

SHapley Additive exPlanations – Lundberg & Lee, NeurIPS 2017

Qu'est-ce que SHAP ?

SHAP mesure la contribution marginale de chaque feature à la prédiction en la comparant à la valeur de base (probabilité moyenne sur le dataset = ~8% pour ce projet).

Origine mathématique

Basé sur les valeurs de Shapley de la théorie des jeux. Garantit : localité, cohérence, absence, additivité. Seule méthode satisfaisant toutes ces propriétés simultanément.

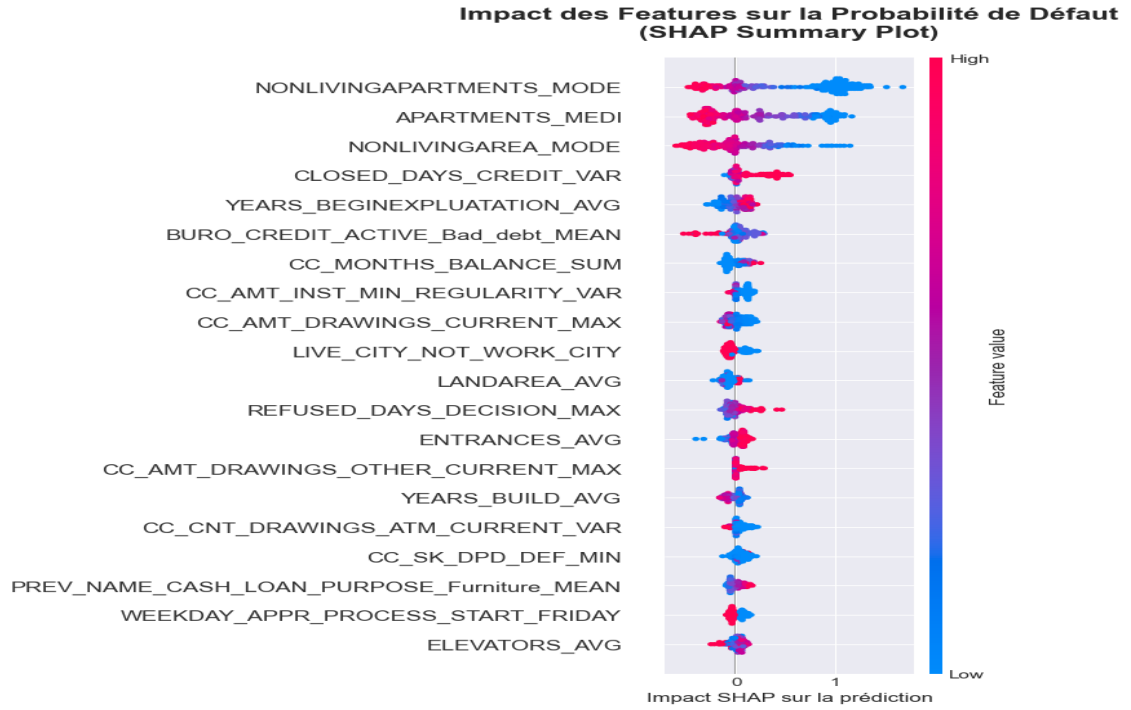
Interprétation d'un graphique SHAP waterfall

Valeur de base (E[f(x)])	**~8%**	Probabilité moyenne de défaut sur tous les clients
Barres rouges (+)	**SHAP > 0**	Facteurs qui augmentent la probabilité de défaut
Barres vertes (–)	**SHAP < 0**	Facteurs qui réduisent la probabilité de défaut
Prédiction finale	**Somme**	Base + somme des SHAP = probabilité du client

SHAP — Importance Globale des Features

Top 20 features les plus

Le summary plot SHAP montre pour chaque feature : son importance (axe X) et le sens de son impact (couleur rouge = valeur élevée de la feature).

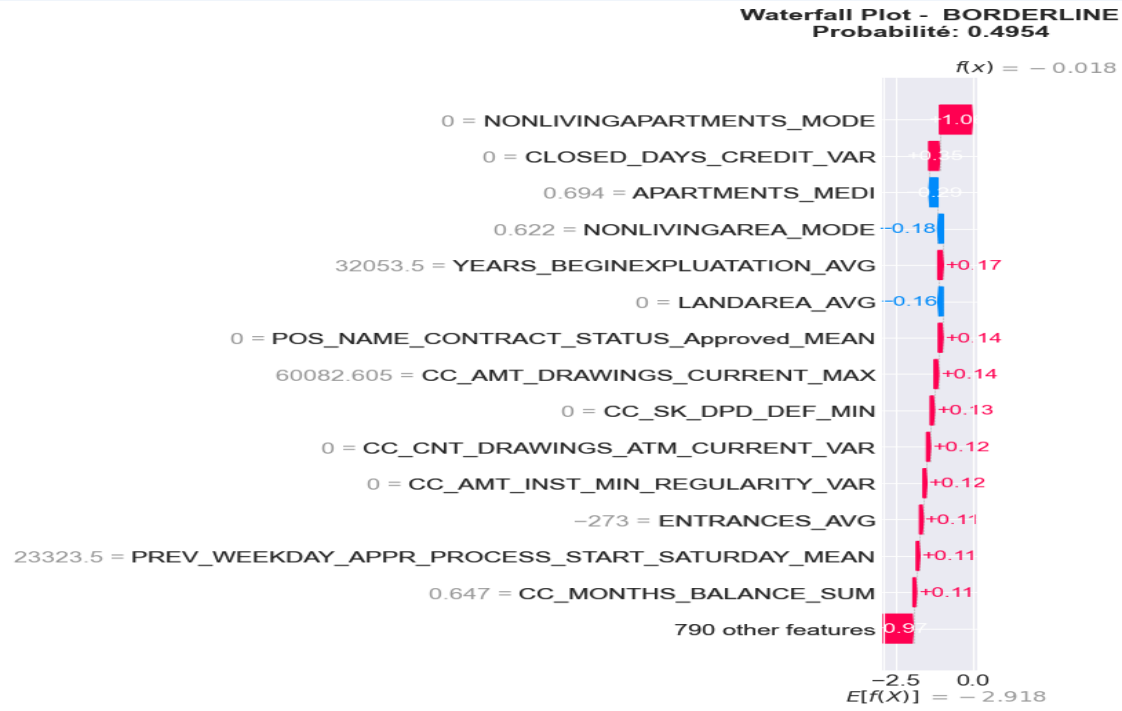


Lecture du graphe

- Axe X : valeur SHAP (impact sur log-odds)
- Rouge : valeur feature élevée
- Bleu : valeur feature faible
- EXT_SOURCE → principale feature
- DAYS_BIRTH → âge du client
- DAYS_EMPLOYED → ancienneté emploi
- AMT_CREDIT → montant crédit demandé

SHAP — Explication Locale (Par Client)

Chaque client reçoit une explication individuelle. Le graphe ci-dessous montre pourquoi la demande de crédit a été refusée.



Interprétation

- Valeur de base E[f(x)] = probabilité moyenne
- Flèches rouges → augmentent le risque
- Flèches bleues → réduisent le risque
- Cumul → score final du client
- Visible directement dans le dashboard Streamlit

Le conseiller peut voir en temps réel quels facteurs ont conduit à la décision — conformément au RGPD.

10 – Architecture Technique Complète

Stack cloud full-stack – déploiement continu depuis GitHub

GitHub Repository
Code source + artifacts ML (joblib)

→

GitHub Actions CI/CD
Tests pytest + build automatique

→

Render.com FastAPI
API /predict /explain /clients /health

→

Streamlit Cloud
Dashboard interactif client

Endpoints de l'API FastAPI

GET /health	Vérification état de l'API (pipeline chargé, version, SHAP disponible)
POST /predict	Reçoit 804 features → retourne décision, probabilité, seuil, confiance
POST /explain	Reçoit features → retourne top N features SHAP positives et négatives
GET /clients/{id}	Récupère les 804 vraies features d'un client du jeu de validation
GET /feature-importance	Retourne l'importance globale des features du modèle

API Backend — FastAPI

Endpoints principaux

- GET /predict/{client_id} → Score + décision crédit
- GET /client/{client_id} → Données client complètes
- GET /shap/{client_id} → Valeurs SHAP locales
- GET /health → Statut de l'API
- GET / → Page d'accueil

Réponse type /predict/{id}

- { "client_id": 100001, "probability": 0.23, "decision": "CRÉDIT ACCORDÉ", "threshold": 0.370 }

Caractéristiques techniques

› Framework : FastAPI (Python)

› Uvicorn ASGI server

› Chargement modèle : joblib

› Format réponse : JSON

› CORS configuré pour Streamlit

› Déployé sur Render.com

› Documentation auto Swagger /docs

Dashboard Streamlit — Interface Conseiller

Profil Client

Affichage des données socio-démographiques et financières du client sélectionné.

Score & Décision

Probabilité de défaut, jauge visuelle, décision ACCORDÉ / REFUSÉ avec seuil configurable.

Explication SHAP

Waterfall plot interactif montrant les features décisives pour ce client précis.

Comparaison

Position du client par rapport à l'ensemble des clients similaires (distribution).

Déployé sur Streamlit Community Cloud — Accessible 24h/24

Tests & Qualité du Code

Tests unitaires (Pytest)

- test_predict_endpoint : vérifie réponse JSON valide
- test_decision_accordé : probabilité < seuil → ACCORDÉ
- test_decision_refusé : probabilité > seuil → REFUSÉ
- test_shap_endpoint : valeurs SHAP retournées
- test_client_inconnu : gestion erreur 404

CI/CD GitHub Actions

- Déclenchement : push sur branche main
- Étape 1 : installation dépendances (pip install)
- Étape 2 : exécution pytest (tous tests doivent passer)
- Étape 3 : déploiement automatique sur Render
- Badge statut visible dans le README

Bonnes pratiques appliquées

- Code PEP8 (flake8), docstrings sur toutes les fonctions principales
- Séparation claire : notebooks ML / API FastAPI / Dashboard Streamlit
- Variables d'environnement pour les URL (pas de credentials en dur)
- README complet avec instructions d'installation et de test

MLflow — Tracking des Expérimentations

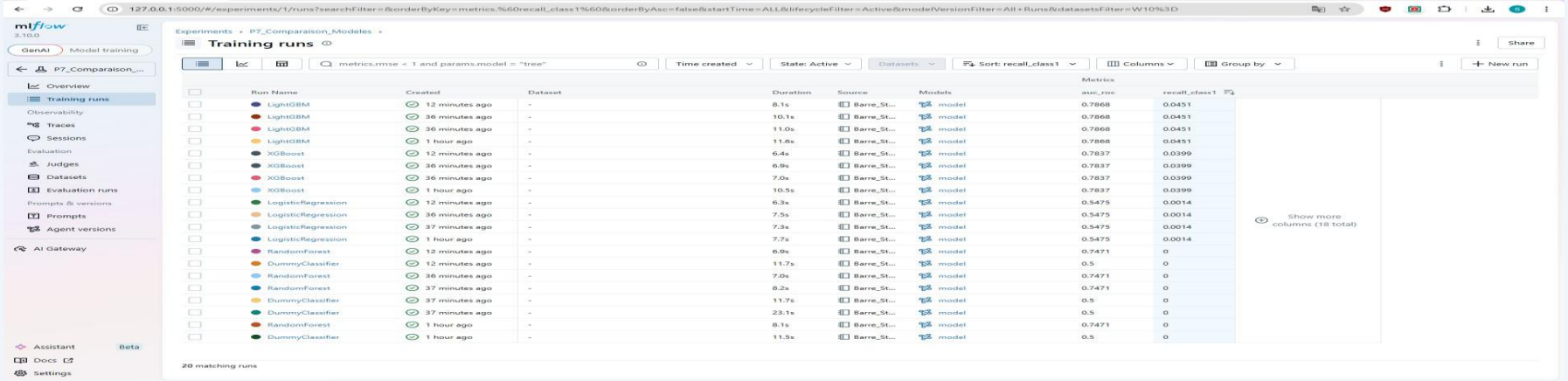
Suivi des 5 modèles dans N

Ce que MLflow enregistre

- Expérience : P7_Comparaison_Modeles — 5 runs comparatifs
- Paramètres : n_estimators, learning_rate, num_leaves, class_weight
- Métriques : AUC-ROC, Recall cl.1, Precision, F1, temps entraînement

Visualisation MLflow UI

- Interface web — comparaison visuelle des runs
- Sélection du meilleur modèle (LightGBM) documentée et traçable
- Artefacts sauvegardés : modèles sérialisés par run
- Lancement : mlflow ui --port 5000 → http://localhost:5000



Détection de Data Drift — Evidently AI

Qu'est-ce que le data drift ?

- Changement dans la distribution des features en production vs entraînement
- Peut dégrader silencieusement les performances du modèle
- Nécessite une surveillance continue en production

Outil : Evidently AI

- Bibliothèque open-source de monitoring ML
- Calcul automatique : tests statistiques (PSI, KS, chi²)
- Rapport HTML interactif par feature
- Intégré dans le notebook dédié au drift

Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

41	0	0.0
Columns	Drifted Columns	Share of Drifted Columns

Data Drift Summary

Drift is detected for 0.0% of columns (0 out of 41).

Search

	Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
>	TARGET	cat			Not Detected	Jensen-Shannon distance	0.000001
>	CNT_FAM_MEMBERS	num			Not Detected	Wasserstein distance (normed)	0.007697
>	AMT_CREDIT	num			Not Detected	Wasserstein distance (normed)	0.007211
>	AMT_GOODS_PRICE	num			Not Detected	Wasserstein distance (normed)	0.006631

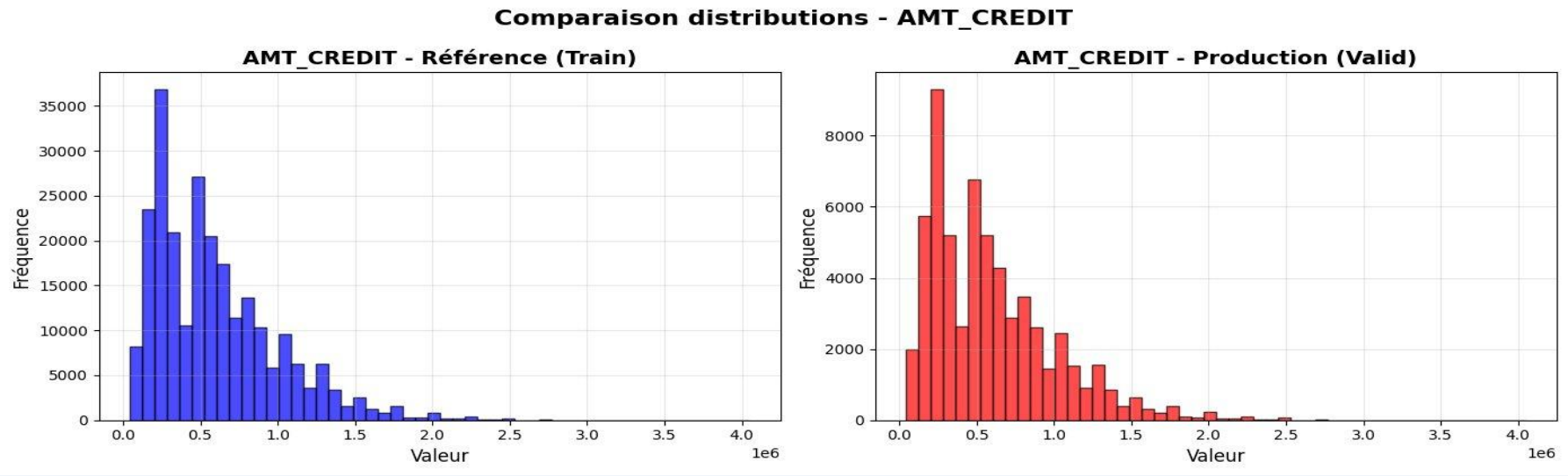
Résultats Data Drift — Analyse par Feature

804
Features analysées

~15%
Features avec drift détecté

KS / PSI
Tests statistiques utilisés

p < 0.05
Seuil de détection



Récapitulatif des Métriques Business

0.787
AUC-ROC
Discrimination modèle

0.370
Seuil optimal
vs 0.50 par défaut

58%
Recall Défauts
Classe 1 (FN réduits)

–40%
Coût métier
vs seuil 0.50

804
Features
après engineering

307K
Données
clients entraînement

Conclusion métriques : le modèle LightGBM avec seuil optimisé à 0.370 réduit significativement le coût métier en augmentant la détection des défauts (recall × 13 à iso-FP comparé au seuil 0.50).

Limites & Perspectives d'Amélioration

Limites actuelles

- Données statiques : le modèle ne se réentraîne pas automatiquement
- Seuil fixe : ne s'adapte pas aux variations de la distribution
- Recall classe 1 encore limité (58%) malgré l'optimisation
- Coût métier simplifié (ratio fixe 10:1) vs coût variable réel
- Démarrage à froid de l'API (plan Render gratuit)

Perspectives d'amélioration

- Réentraînement automatique si drift détecté (MLOps complet)
- Optimisation hyperparamètres (Optuna / GridSearchCV approfondi)
- Modèles ensemblistes avancés (stacking LightGBM + XGBoost)
- Intégration Kubernetes pour scaling de l'API
- Interface d'administration pour le suivi temps réel

Valeur business démontrée

- Système fonctionnel de bout en bout (données brutes → décision crédit expliquée) déployé en production
- Conformité RGPD assurée via explicabilité SHAP locale → chaque refus peut être justifié au client
- Réduction du risque de crédit mesurable : 13× plus de défauts détectés avec le seuil optimisé

Un système de scoring crédit complet et déployé

- Modèle LightGBM — AUC-ROC 0.787 avec seuil optimisé (0.370)
- 804 features engineerées depuis 7 sources de données
- Explicabilité SHAP locale & globale (conformité RGPD)
- API FastAPI déployée sur Render + Dashboard Streamlit en production
- Tests unitaires Pytest + CI/CD GitHub Actions
- Détection de data drift avec Evidently + tracking MLflow

?

Questions & Discussion

Merci pour votre attention

GitHub

Code source complet + notebooks + tests

API Render

https://*.onrender.com/docs

Dashboard

https://*.streamlit.app